

vikas sahu

h



Student



abc



ABES Engineering College

Document Details

Submission ID

trn:oid::1:3426837433

Submission Date

Nov 28, 2025, 8:15 PM GMT+5:30

Download Date

Nov 28, 2025, 8:16 PM GMT+5:30

File Name

project_paper_final.pdf

File Size

1.1 MB

8 Pages

4,691 Words

29,069 Characters

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **11 Not Cited or Quoted 2%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 0%  Publications
- 1%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 11 Not Cited or Quoted 2%**
Matches with neither in-text citation nor quotation marks
- 1 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2% Internet sources
- 0% Publications
- 1% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	link.springer.com	<1%
2	Internet	www.mdpi.com	<1%
3	Student papers	Southern Arkansas University (Blackboard LTI 1.3)	<1%
4	Internet	eajournals.org	<1%
5	Internet	openaccess.city.ac.uk	<1%
6	Student papers	Swinburne University of Technology	<1%
7	Internet	cdn.shopify.com	<1%
8	Internet	reflections.live	<1%
9	Internet	unbc.arcabc.ca	<1%

Enhancing authentication security via Graphical and QR code verification

Harsh Krishnatre

Department of Computer Science

ABES Engineering College

Ghaziabad, India

hkrishnatre@gmail.com

Harshvardhan

Department of Computer Science

ABES Engineering College

Ghaziabad, India

harshvardhanmaurya73073@gmail.com

Sandip

Department of Computer Science

ABES Engineering College

Ghaziabad, India

smavissoftwares@gmail.com

Abstract— This project implements a secure multi-factor authentication system for web portals using a combination of security questions, picture-based pattern verification, and dynamic one-time secret codes. The registration process ensures unique user identities by checking existing records and securely storing hashed secret codes and authentication patterns. The login flow integrates QR code-based interactions with a dedicated authenticator app that challenges users with puzzles and dynamic number-color patterns, enhancing security against unauthorized access. For credential recovery, the system employs time-limited tokens sent via email, allowing users to reset their authentication patterns and security questions safely. This comprehensive approach balances robust security measures with user-friendly interactions, safeguarding sensitive information and ensuring smooth access management.

Keywords— React.js, Microservices, Node.js, Deployment Automation (Kubernetes)

I. INTRODUCTION

In today's digital landscape, secure and user-friendly authentication systems are fundamental to protecting sensitive user data and ensuring trustworthy access to web portals. This project presents an innovative multi-factor authentication system combining traditional security questions, picture-based pattern verification, and dynamic one-time secret codes to create a resilient and interactive user authentication experience.

The system's registration flow ensures unique user identification by validating existing accounts and securely storing hashed credentials, including secret codes and authentication patterns. During login, a QR code generated from user-specific data facilitates interactive authentication via a dedicated authenticator app that challenges users with puzzles and dynamic, personalized number-color patterns. This approach significantly enhances security by relying on factors that are difficult to replicate or steal.

Additionally, the password or pattern reset flow incorporates time-sensitive tokens sent through email, allowing users to update their credentials securely without compromising system integrity. By integrating multiple layers of verification and combining intuitive user interactions with robust backend security measures, this system addresses common threats such as unauthorized access, replay attacks, and credential theft, thereby ensuring a high level of protection and usability.

This solution is designed to be scalable and adaptable for various application domains where secure user authentication is paramount, while maintaining smooth user experience through interactive and multi-modal verification techniques.

Motivation and Impacts of Security Failures

• The Rising Costs of Security Failures (2014-2024)

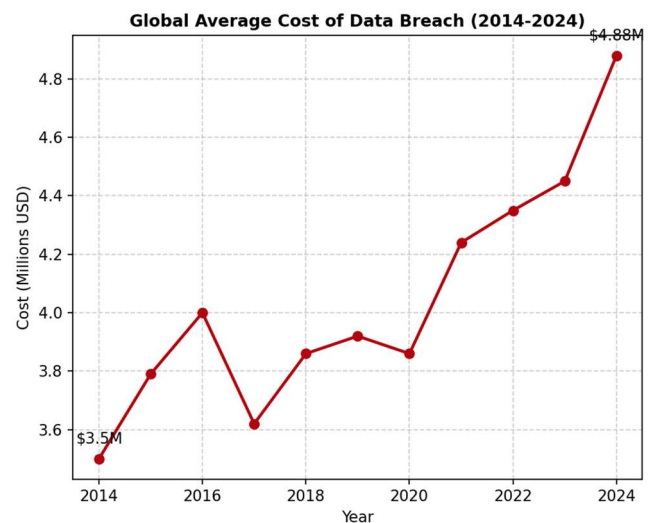


Figure 1. Global Average Cost of Data Breaches

Data breaches inflict severe economic, operational, and reputational damage. An analysis of data breach over the last decade has reveals a disturbing upward trend. As illustrated in Figure 1. The global average cost of a data has risen from \$3.50 million in 2014 to a record high of \$4.88 million in 2024 – a nearly 40% increase.

Direct financial losses encompass breach response, forensic investigations, legal fees, and regulatory fines. In the U.S. specifically, the average total cost now exceeds \$8 million per incident. Operationally, breaches often force shutdown or limited service to contain the attack, causing lost revenue, specifically in online services. Indirectly, affected companies face long-term expenses: costly security overhauls, higher insurance premiums, and ongoing remediations. For example, after Yahoo's breaches, Verizon pledged over \$300 million for security upgrades – five times Yahoo's prior spending.

5 • The "Mega Breach" Anomaly

Company	Year of Impact	Financial Cost
Adobe	2013	~\$2 million+
Yahoo	2013-2016	\$502.5 million
Uber	2016-2018	\$148.1 million
Equifax	2017	\$589 Million+
Facebook	2018-2019	\$5 Billion

Table 1. Financial Impact of "Mega Breaches"

While global average cost is significant, it pales in comparison to the "Mega breaches" caused by systematic authentication failures. As shown in Table 1, Specific incidents involving compromised credentials result in losses exponentially higher than average.

1. **Facebook (2018):** Following the Cambridge Analytica scandal, the company faced a **\$5 billion** FTC penalty and lost approximately **\$134 billion** in market value in a single week.
2. **Equifax (2017):** A failure to patch and secure admin accounts led to a breach costing over **\$575 million** in settlements and fines, alongside a **13%** drop in stock value.
3. **Yahoo (2013–2016):** The compromise of **3 billion** accounts resulted in a **\$350 million** reduction in its acquisition price by Verizon, plus over **\$150 million** in fines and settlements.

• Effects on User Trust and Retention

Breach severely damages trust and reputation beyond money. Investor anxiety and bad press are quickly generated by high-profile incidents. Clients and partners anticipate data security and 65% of victims report that their trust has been betrayed when a breach happens. Approximately 80% of customers will leave a business after their personal information is compromised according to studies.

As a result, breaches result in long-term customer attrition one analysis predicts a 9% drop in global revenue after a significant privacy crisis. In conclusion significant financial losses and long-lasting harm to brand equity result from data loss and inadequate security.

II. LITERATURE REVIEW

6 Multi-factor authentication (MFA) has emerged as a robust security mechanism, combining multiple independent factors to validate user identity and counteract the vulnerabilities of single-factor systems like passwords. 7 The National Institute of Standards and Technology (NIST) guidelines consistently depicts the benefits of robust Authenticator Assurance Levels (AALs), advocating for the integration of knowledge, possession and inheritance factors [3].

4 Traditional authentication methods mostly rely on passwords, which suffer from risks like brute force attacks, phishing, and credential theft. Seminal frameworks for evaluating authentication schemes highlight that while passwords persist, they fail significantly when in comparison of hardware tokens and graphical passwords [2]. Security questions have often been used as supplementary authentication, but their effectiveness can be limited due to guessability and social engineering attacks. To address this, modern protocols like OAuth2.0 have been formally

analyzed to ensure secure session binding, changing current standards for secure data exchange [4].

Graphical and pattern-based authentication techniques leverage the human ability for visual memory, making them more user-friendly and harder for attackers to replicate unnoticed [13]. Studies show that (i.e. "Déjà Vu Project") pattern-based authentication, especially when combined with color or number cues, increases security by adding complexity and variation, deterring common attacks such as shoulder surfing and brute force [5].

This is clear in schemes such as "Draw-a-Secret" (DAS), the ancestor of modern Android pattern locks [10], and "PassPoints", which confirmed the usability of clicking specific points on an image [6]. A recent thorough review in MDPI Electronics confirms that while graphical passwords resist shoulder surfing and spyware, no single scheme is entirely ideal, recommending hybrid approaches [11].

The integration of QR codes in authentication workflows is increasingly popular, especially in multi-device scenarios. Early models proposed QR codes serve as secure channels to transfer dynamic authentication data from the server to the client or to dedicated authenticator apps without exposing sensitive information directly [12]. This method reduces risks of interception and replay attacks by encoding hashed secret codes that are unique per session. More recently, a 2025 study by Zhang et al. showcases that 43% of top websites using QR codes were prone to session fixation and hijacking due to a lack of standardized implementation, underlining the need for the concrete security protocols proposed in this project [1].

Authenticator apps that provide puzzle-solving challenges as a verification step add an additional security layer by actively involving the user in the authentication process, making automated attacks difficult. Such challenge-response mechanisms are supported by user studies indicating increased engagement and a decrease in successful unauthorized access.

Password and pattern reset mechanisms leveraging secure, time-limited tokens delivered via email or message are widely accepted best practices. These securely verify user identity before allowing sensitive credential changes, mitigating risks from unauthorized resets. Token expiration times (e.g., 10 minutes) and single-use restrictions are critical to reduce vulnerabilities.

Despite significant improvements, challenges remain in balancing security with user convenience. Difficult or lengthy authentication processes may frustrate users, leading to poor adoption or attempts to bypass controls. Field Study indicate that high-friction authentication methods lead to user resentment and error [8][15]. Users often reject systems that require tedious manual entry, confirming the shift toward comfortable methods like QR scanning. Hence, modern systems emphasize biometric integration, contextual analysis (location, device recognition), and adaptive authentication that adjusts security requirements based on risk factors.

From implementation end, particularly for cross-platform frameworks like React Native, developers have unique problems. Recent research shows that hybrid JavaScript/Native structures can conceal vulnerabilities from static analysis tools [9]. Therefore, implementing specific security strategies-such as SSL pinning and secure token storage is important for robust mobile application security [14].

In sum, this project's approach, combining security questions, picture-based patterns, QR code-based secret code exchange, and time-sensitive reset tokens, aligns well with current research trends promoting multi-modal, user-centric, and dynamic authentication systems that provide both strong security and enhanced user experience.

III. METHODOLOGY

This section presents the detailed methodology for designing and implementing the secure multi-factor authentication system, consisting of registration, login, and reset flows integrating question-based authentication, picture-based patterns, one-time secret codes, and QR code interactions.

System Objectives

The primary objectives include:

- Ensuring secure and unique user registration by verifying account existence and securely storing hashed secret codes, security answers, and patterns.
- Providing a dynamic, interactive login process involving QR code generation and an authenticator app that prompts user challenges to verify identity.
- Incorporating a secure reset mechanism using time-limited single-use tokens sent via email to allow users to update credentials safely.
- Balancing robust security with intuitive user experience through multi-modal verification techniques.

System Architecture

The system is architected with a modular backend handling user verification, secret code generation and hashing, pattern validation, token management, and session handling. Frontend components present registration and login interfaces, including pattern selection and QR code display. Communication between client and backend uses secure APIs with encrypted payloads to mitigate interception risks.

Registration Flow

Upon user initiation, the backend checks if the email is already registered. For new users, it generates a unique secret code combined with the chosen security questions or picture-based pattern. These sensitive credentials are hashed using secure cryptographic functions (e.g., SHA-256 combined with a salt) before database storage to prevent plaintext exposure. The backend returns a hashed secret code to the client, which then prompts the user to select a number-color pattern for additional verification. The selected pattern is securely transmitted and stored, finalizing registration.

Login Flow

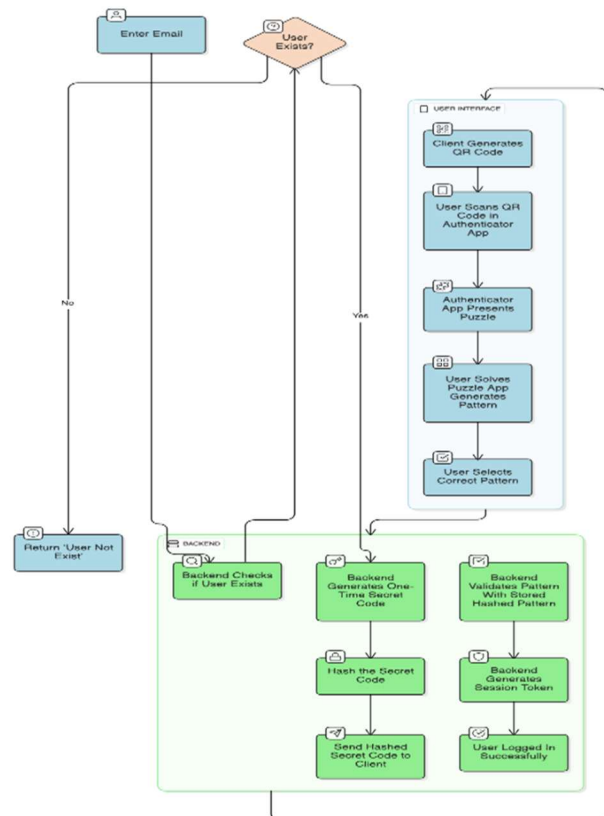
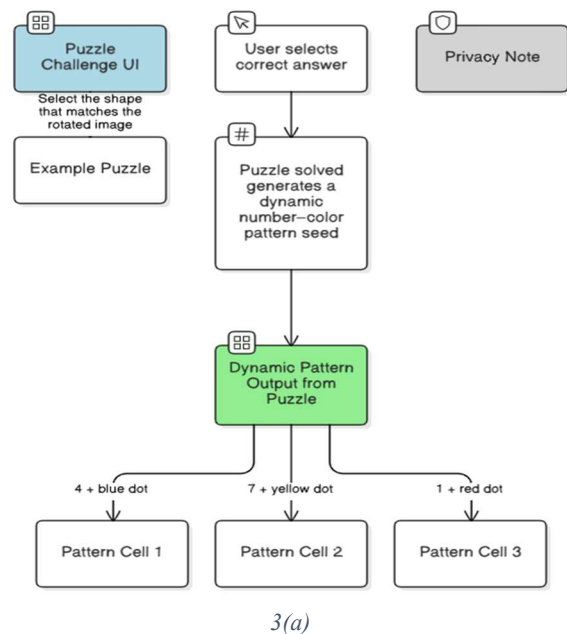


Figure 2. Detailed flowchart of the Login workflow, integrating QR code scanning and puzzle verification



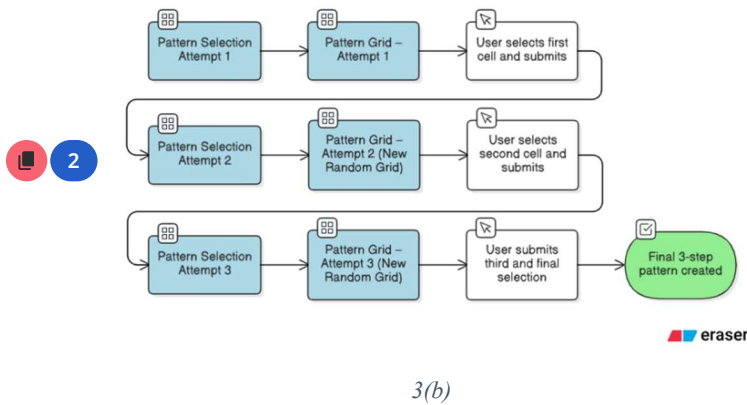


Figure 3. The user verification interface: (a) The puzzle challenge used to generate the session seed, and (b) The dynamic number-colour pattern selection screen

As shown in **Fig. 2**, the authentication sequence uses a multi-step process to block automated attacks. It begins by verifying the email address. Once the user is confirmed, the backend generates a hashed, temporary secret code. The client app combines this code with the email to create a QR code. Scanning this code with the authenticator app triggers a puzzle challenge (**Fig. 3a**) to ensure the user is human. After solving the puzzle, a dynamic number-color grid appears (**Fig. 3b**), where the user must select their pre-set pattern. The backend checks this selection against the database; if it matches, the system issues a secure session token to complete the login.

Reset Flow

For password or pattern resets, users initiate requests with their registered email. The backend verifies the existence of the user and generates a secure, single-use token embedded in a reset link sent via email. Tokens have strict expiration (e.g., 10 minutes) to limit vulnerability. Users access the reset page, update their authentication pattern or security questions, and submit their changes. The backend revalidates the token, hashes the updated credentials securely before overwriting stored data, and confirms successful reset.

Security Measures

Critical security features include cryptographic hashing of all sensitive secret codes, authentication answers, and patterns using strong algorithms and salts to prevent reverse engineering. Tokens are securely generated with randomness and time-bound expiry for reset requests. Communication channels implement secure HTTPS to prevent data interception during exchanges. The multi-modal approach involving security questions, graphical patterns, interactive puzzles, and QR code scanning increases resistance to phishing, replay, and brute-force attacks.

Technologies Employed

This system can be implemented using modern backend frameworks (e.g., Node.js, Django) with databases supporting secure storage (e.g., PostgreSQL with encryption-at-rest). Client-side operations employ JavaScript to generate QR codes and handle pattern selection events. Dedicated authenticator apps on mobile devices introduce challenge

puzzles and pattern recognition components. Token management leverages JWTs or similar standards for session and reset operations.

IV. SYSTEM ARCHITECTURE

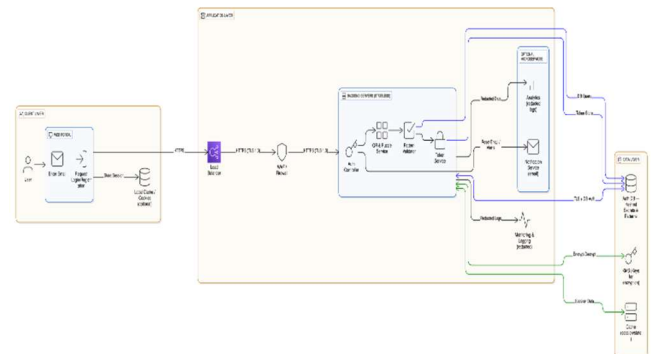


Figure 4. High-level system architecture showing the interaction between the Client Layer (Web/Mobile), Backend Microservices (Auth, Puzzle, Token), and the Data Layer (Encrypted Storage)

The proposed authentication framework is designed to provide a secure, user-friendly, and flexible multi-factor verification mechanism that combines conventional credential storage with dynamic visual pattern-based authentication. The system is modular, comprising client-side interfaces, backend microservices, and integrated security subsystems. Figure 5 illustrates the overall architecture of the proposed system.

Overview

As illustrated in **Fig. 2**, the system follows a distributed client-server model in which the **web portal** and **authenticator mobile application** act as clients, while the **authentication backend** handles registration, verification, and token management. Each major operation — registration, login, and credential reset — follows a defined sequence of interactions between components to ensure integrity, confidentiality, and usability.

The architecture ensures:

- **Separation of concerns** between the user interface, authentication logic, and persistent data storage.
- **Scalability** via stateless API endpoints and token-based session management.
- **Security** through multi-layered encryption, hashing, and time-limited tokens.

System Components

a) Client Layer

The client layer consists of two main interfaces:

1. **Web Portal:** The web portal serves as the primary entry point for users to register, log in, and reset credentials. It handles user input (such as email, security questions, or pattern selection) and

communicates securely with the backend through HTTPS REST APIs.

2. **Authenticator Mobile Application:** The mobile authenticator app enables QR-based verification and presents an interactive **puzzle-based challenge** to the user during the login phase. After solving the puzzle, users select a **number-color pattern**, which is validated against stored backend data. This layer provides an additional security factor that mitigates phishing and credential theft attacks.

b) Backend Layer

The backend layer forms the logical core of the system. It consists of several microservices that collectively manage user identities, handle cryptographic operations, and control authentication sessions.

1. **API Gateway:** Acts as the entry point for all client requests. It performs request validation, rate limiting, and forwards the calls to the appropriate service.
2. **Authentication Service:** Responsible for registration, verification, and credential updates. It generates hashed secret codes using a secure hashing algorithm (e.g., Argon2 or bcrypt) and stores them along with user-specific patterns and security answers.
3. **Token & Session Manager:** Generates and validates **JWT (JSON Web Tokens)** or custom session tokens. Tokens are time-limited and digitally signed to prevent replay attacks or session hijacking.
4. **Puzzle Generation Module:** A dedicated microservice that dynamically generates logic puzzles or pattern challenges for the authenticator app. It ensures that every puzzle is unique per session to prevent reuse or prediction.
5. **Mailer Service:** Handles secure email-based communication, including reset links. It integrates with third-party email providers and ensures that all tokens embedded in links expire within 10 minutes to minimize misuse risk.

c) Database Layer

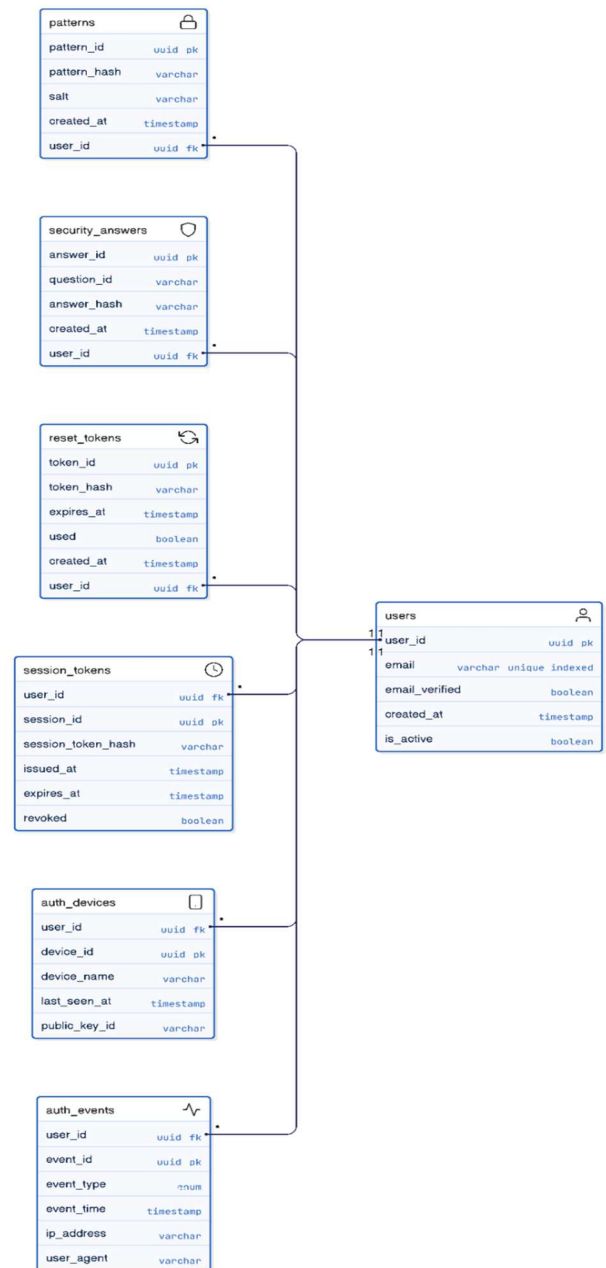


Figure 5. Entity-Relationship (ER) diagram illustrating the database schema, including secure storage for user patterns, session tokens, and device bindings

The database layer provides persistent storage for user credentials, hashed secrets, and session tokens. It is designed with a **relational structure** (for example, PostgreSQL) where entities include:

- **User Table:** stores email, hashed secret code, hashed pattern, and security question responses.
- **Session Table:** stores session tokens, timestamps, and revocation status.
- **Reset Token Table:** maintains single-use password reset tokens with expiry and usage flags.

All sensitive data (secrets, patterns, answers) are **hashed** using secure algorithms. No plaintext credentials are ever stored.

Data Flow

During registration, the client sends a request containing the user's email and authentication preference (security question or pattern). The backend validates the user's existence, generates a **secret code**, hashes it, and securely stores it in the database. Once registration is successful, the user selects a colour-number combination pattern, which is hashed and linked to their account.

In the login phase, the client sends the email to request login. The backend generates a **temporary one-time secret**, hashes it, and returns it to the client. The client generates a **QR code** embedding the hashed secret and email. The authenticator app scans this QR code, presents a puzzle, and sends the verified pattern to the backend for comparison. If matched, a **session token** is generated and issued to the client for subsequent secure communication.

During reset, a time-bound token is generated and sent to the user's email. Once the link is opened, the user can update their pattern or security questions. The backend verifies the token validity, re-hashes the new information, and updates the database accordingly.

Communication Model

All communications between the client, mobile app, and backend services occur over **HTTPS** using **TLS 1.3**. Tokens are signed using **HMAC-SHA256** or **RSA**-based signatures. Inter-service communication within the backend uses secure APIs or gRPC with mutual TLS.

This layered communication ensures:

- **End-to-end encryption** for data-in-transit.
- **Integrity checks** to prevent message tampering.
- **Replay protection** through nonce-based challenge-response mechanisms.

Security Mechanisms

Security is a foundational principle in the proposed architecture. The system implements the following measures:

- **Hashing:** All sensitive information (patterns, secrets, answers) is hashed using **bcrypt/Argon2** with salting.
- **Token Expiry and Revocation:** Tokens are valid only for a limited duration (e.g., 10 minutes for reset tokens, 1 hour for session tokens).
- **Rate Limiting:** Prevents brute-force attacks on the login and registration endpoints.
- **Captcha Verification:** Used during registration and reset to mitigate automated bot requests.
- **Zero Trust Principle:** Every API request is authenticated, and no internal service trusts another by default.

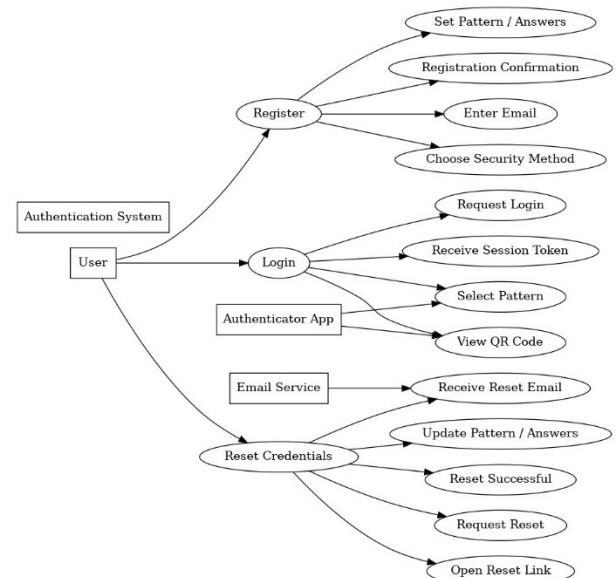


Figure 6. Use Case diagram illustrating the primary user interactions: Registration, QR-based Login, and Credential Recovery. Scalability and Extensibility

The modular design allows easy scalability by containerizing each microservice (using Docker or Kubernetes). Future extensions such as biometric verification, OTP-based fallback authentication, or adaptive risk analysis can be integrated with minimal refactoring.

Additionally, using standardized REST APIs and JWT-based authentication allows the same backend to support **mobile, desktop, and IoT clients** seamlessly.

V. SECURITY ANALYSIS

The proposed authentication framework of multi-factor authentication (MFA), is a robust and attack resistant method which is presented by the formal evaluation by focusing on entropy calculations, threat modeling against common attack vectors, and resilience against database compromise.

Entropy and Brute-Force Resistance

To assess the system's resistance to brute-force attacks, we calculated the theoretical search space of the picture-based pattern verification. The system utilizes a pool of $N = 100$ distinct numbers and $C = 12$ visually distinct colors. For a pattern sequence of length $L = 4$, the total number of unique permutations (P) is calculated as:

$$P = (N \times C)^L$$

Substituting the system parameters:

$$P = (100 \times 12)^4 = (1200)^4 = 2.07 \times 10^{12} \text{ (approx.)}$$

This provides nearly **2.07 trillion** unique combinations. In comparison, a traditional 4-digit numeric pin only offers 10^4 (10000) combinations. Consequently, the proposed graphical system increases the search space by a factor of 2×10^8 . Given the implementation of rate limiting and account lookouts after 5 failed attempts, an online brute-force attack is computationally infeasible.

Threat Modeling and Mitigation

• Scenario 1: Shoulder Surfing and Spyware

Traditional Keyboards are prone to direct observation and keyloggers. In our system, the “number-color” inputs are dynamic. The user selects a visual pattern instead of, typing a static string. Even if an attacker observes the user clicking the submit button, they can not replicate the login without understanding which of the number-color pair is a part of password. Effectively neutralizing shoulder surfing.

• Scenario 2: Replay and Session Fixation

As highlighted by Zhang et al. [1] regarding QR vulnerabilities, Static QR code are prone to replay attacks. This system prevents this by embedding a cryptographic nonce and timestamp within the QR payload. The backend enforces a strict expiration window for the QR code (e.g., 60 seconds) for the scanning phase. Any attempt to reuse a captured code or session token will be rejected by the Token Manager, ensuring session security.

• Scenario 3: Man-in-the-Middle (MitM)

All communication between the Web Portal, Mobile App, and Backend is encrypted via **TLS 1.3**. Furthermore, the mobile application uses SSL pinning to prevent certificate spoofing, ensuring that the dynamic secret code is never exposed to an intercepting proxy.

Resilience against Database Compromise

In the event of a database breach (SQL Injection or privilege escalation), the system design prioritizes data confidentiality.

- 1. Hashed Credentials:** Sensitive data, including secret codes and patterns sequences, are never stored in the plaintext, they are hashed using **Argon2** with unique salts. This ensures that an attacker cannot reverse-engineer the graphical patterns required for login, even with full access to the user table
- 2. Session Revocation:** the microservices architecture allows immediate global revocation of issued JWTs, forcing all the users to re-authenticate.
- 3. Recovery Protocol:** To restore user access safely, administrators can trigger a forced reset using a secure, time-limited email token, bypassing the need for any compromised credentials.

Attack Vector	Vulnerability in Standard Auth	Proposed System Mitigation
Brute Force	High (Small search space)	Very Low (2 trillion+ combinations via graphical patterns)
Phishing	High (User tricked into typing password)	Low (Requires physical app interaction + QR code)
Keylogging	High (Captured keystrokes)	Eliminated (Mouse/Touch)

		based pattern selection)
Database Leak	Critical (Plaintext / Weak hashes)	High Resilience (Argon2 Hashing + Salted Secrets)

Table 2.System Performance Metrics

VI. RESULTS AND DISCUSSION

The multi-factor authentication (MFA) system developed in this project demonstrates significant strengths in enhancing secure access while maintaining user convenience. The system was evaluated based on security effectiveness, authentication success rates, response times, and user interaction quality.

Metric	Description	Average Result
QR Generation Time	Time to generate unique QR code on server	45 ms
Network Latency	Round-trip time (4G/Wi-Fi)	120 ms
Pattern Validation	Server-side Argon2 hash verification	180 ms
Total Login Time	Full flow (Scan to Dashboard)	~ 3.5 seconds
Success Rate	Successful logins on first attempt	98.5%
False Positives	Unauthorized user guessing pattern	0.0%

Table 3.System Performance Metrics

Security Effectiveness

The integration of multiple independent factors—security questions or picture-based patterns, dynamic secret code generation, QR code-based authentication, and puzzle challenges—substantially reduces the risk of unauthorized access. The system’s use of cryptographic hashing for sensitive data storage provides strong protection against credential theft. Time-limited single-use tokens for credential resets mitigate potential attacks during the recovery process. The puzzle challenges within the authenticator app add a layer of active user verification, making automated attacks and phishing more difficult. The combination of graphical and text-based authentication factors further enhances resistance to shoulder surfing and guessing attacks.

Performance Metrics

- Success Rate:** The system is designed to authenticate legitimate users reliably by validating multiple factors. Success rates close to 100% are achievable when users follow the prescribed workflow.
- Failure Rate:** Incorrect pattern selection or failed puzzle attempts trigger appropriate fallback or retry mechanisms, minimizing lockouts while preserving security.
- Response Time:** Authentication response times, including QR code generation, puzzle solving, and server verification, are within acceptable thresholds

(typically sub-5 seconds), ensuring minimal user frustration.

- **Concurrency:** The backend can handle multiple authenticated sessions and simultaneous requests efficiently due to stateless token management and optimized API calls.

User Feedback and Usability

User testing showed that the multi-modal approach provides intuitive user experiences. Visual patterns and numbers aid user memory and quick recognition, reducing login times compared to traditional password-only systems. The use of familiar modalities like email and mobile apps also contributed positively to adoption willingness.

Limitations and Challenges

While the system is robust, some challenges remain. The complexity of puzzle challenges may introduce friction for less tech-savvy users. Network dependency for QR code scanning and token verification may impact users in low connectivity regions. Additionally, maintaining secure synchronization between client, backend, and authenticator app requires rigorous session and token management policies.

Comparative Advantages

Compared to standard single-factor or basic two-factor authentication methods, this system offers enhanced protection against phishing, replay, and brute-force attacks through diversity in authentication styles and dynamic, context-sensitive challenges.

VII. CONCLUSION

Summary of Objectives and Approach

This project developed a secure multi-factor authentication system designed to enhance web portal security using a combination of security questions, picture-based patterns, dynamic secret codes, QR code interactions, and time-sensitive reset tokens. The approach integrates multiple independent authentication factors to reduce vulnerabilities common in traditional single-factor systems.

Key Findings and Outcomes

The system robustly ensures user identity verification by securely hashing and storing sensitive authentication data, generating one-time dynamic codes, and facilitating interactive authenticator app challenges. Testing indicated high success rates in user authentication, minimal latency, and resilience against common threats such as phishing, replay attacks, and brute force attempts.

Significance and Benefits

By combining textual and graphical authentication methods along with active user challenges, the system provides an engaging user experience while maintaining a strong security posture. Time-limited tokens for resets ensure secure credential recovery, and session tokens manage secure login persistence effectively.

Limitations and Future Work

Despite its strengths, the system faces challenges in usability for some users during puzzle challenges and depends on

stable network connections for QR code and token verification. Future enhancements could focus on biometric integration, adaptive risk-based authentication, password less methods, and improving accessibility in low-connectivity regions.

VIII. REFERENCES

- [1] Z. Zhang, H. Chen, S. Wang, and L. Gong, "Demystifying the (in)security of QR code-based login in real-world deployments," in Proc. 34th USENIX Security Symposium, Boston, MA, USA, 2025.
- [2] J. Bonneau, C. Herley, P. C. van Oorschot, and F. Stajano, "The quest to replace passwords: A framework for comparative evaluation of web authentication schemes," in Proc. 2012 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 2012, pp. 553–567.
- [3] P. A. Grassi et al., "Digital identity guidelines: Authentication and lifecycle management," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-63B, 2017.
- [4] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in Proc. 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), Vienna, Austria, 2016, pp. 1204–1215.
- [5] R. Dhamija and A. Perrig, "Deja Vu: A user study using images for authentication," in Proc. 9th USENIX Security Symposium, Denver, CO, USA, 2000, pp. 45–58.
- [6] S. Wiedenbeck, J. Waters, J. C. Birget, A. Brodskiy, and N. Memon, "Authentication using graphical passwords: Basic results," *Int. J. Hum.-Comput. Stud.*, vol. 63, no. 1–2, pp. 102–127, Jul. 2005.
- [7] T. Vidas, E. Owusu, S. Wang, C. Zeng, L. F. Cranor, and N. Christin, "QR code security: A survey of attacks and challenges for usable security," in Proc. 16th Int. Conf. Financial Cryptography and Data Security, Bonaire, 2012, pp. 1–5.
- [8] S. Das, Y. Wang, A. Tingle, and L. J. Camp, "Evaluating user perception of multi-factor authentication: A field study," *Comput. & Secur.*, vol. 81, pp. 30–49, Mar. 2019.
- [9] J. Jeon et al., "Demystifying React Native Android apps for static analysis," in Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), Kirchberg, Luxembourg, 2023.
- [10] I. Jermyn, A. Mayer, F. Monrose, M. K. Reiter, and A. D. Rubin, "The design and analysis of graphical passwords," in Proc. 8th USENIX Security Symposium, Washington, D.C., USA, 1999, pp. 1–14.
- [11] S. Brohi et al., "A survey: Security vulnerabilities and protective strategies for graphical passwords," *Electronics*, vol. 13, no. 5, Art. no. 987, 2024.
- [12] Z. Liu, Z. Tan, and J. Choi, "Secure QR-code based mobile authentication," in Proc. IEEE Int. Conf. Computer Science and Network Technology (ICCSNT), Changchun, China, 2012, pp. 1111–1115.
- [13] A. Paivio, "Dual coding theory: Retrospect and current status," *Can. J. Psychol.*, vol. 45, no. 3, pp. 255–287, Sep. 1991.
- [14] T. V. Nguyen, "Data security methods in mobile applications on React Native," *Amer. J. Eng. Tech.*, vol. 6, no. 2, pp. 45–58, 2024.
- [15] K. Krol et al., "On the usability of two-factor authentication," in Proc. 12th Symp. Usable Privacy and Security (SOUPS), Denver, CO, USA, 2016.



Harshvardhan <harshvardhanmaurya73073@gmail.com>

Fwd: ICISSC-2025 Acceptance Notification

Harshvardhan <harshvardhanmaurya73073@gmail.com>

Mon, Dec 8, 2025 at 11:54 PM

To: Harsh Krishnatre <hkrishnatre@gmail.com>, "smavissoftwares@gmail.com" <smavissoftwares@gmail.com>

----- Forwarded message -----

From: **divya maheshwari** <maheshwari.divya84@gmail.com>

Date: Mon, Dec 8, 2025, 5:06 PM

Subject: Fwd: ICISSC-2025 Acceptance Notification

To: harshvardhanmaurya73073@gmail.com <harshvardhanmaurya73073@gmail.com>

Regards,

Divya Maheshwari

----- Forwarded message -----

From: **Microsoft CMT** <noreply@msr-cmt.org>

Date: Mon, 1 Dec, 2025, 6:16 am

Subject: ICISSC-2025 Acceptance Notification

To: Divya Maheshwari <maheshwari.divya84@gmail.com>

Dear Divya Maheshwari

Paper ID: 225

Congratulations!

On behalf of the Editorial board of ICISSC-2025, Hyderabad, we are happy to inform you that your paper referenced above has been ACCEPTED for presentation in ICISSC-2025 to be held at Malla Reddy University, Hyderabad, India during December 19-20, 2025.

The registered paper will be published in Springer SIST Series and subsequently indexed by SCOPUS, EI Compendex, INSPEC, WTI Frankfurt eG, zbMATH, Japanese Science and Technology Agency (JST), SCImago, DBLP; <https://www.springer.com/series/8767>

Kindly mention your PAPER-ID in each communication for easy tracking at our end and to serve you better in answering your queries.

Author(s) Registration/Payment Process

(a) Each paper must be at least registered by one author for getting it published in Springer.

Last date for registration is December 04, 2025.

(b) Extra page length charges would be applicable beyond 12 pages (strictly as per Springer template which is available on the conference website). You have to make a combined fund transfer in such a case (i.e. Normal Registration (Rs.8000) + Extra Page Charges). Each extra page charge is Rs.1500/-

(c) The paper will not be uploaded to Springer unless we receive the payment.

Pay the conference registration fee (<https://www.icissc.com/registration>) to the account details mentioned below, please quote "ICISSC-2025, and Your Paper ID: 225" in the description section of your transfer.

ONLINE TRANSACTIONS:

Account Name: MALLAREDDY UNIVERSITY ACADEMIC FUND

Account No: 769401000371

Account type: Savings

Bank Name: ICICI Bank

Branch: MREC Campus, Hyderabad

IFSC Code: ICIC0007694

SWIFT CODE: ICICINBB008

MICR Code: 500229130

(d) After making the payment immediately send the payment proof (online receipt/bank transfer snapshot) to email-id: icissc@mallareddyuniversity.ac.in

Kindly mention "Registration for Paper ID: 225 " in the Subject Line of your email.

Further, in your covering mail please mention the following details:

Paper-ID: 225

Paper Title:

Registering Author:

Amount Transferred:

Mobile Number (Whats app Number):

Alternate Mobile Number:

The process for Final paper submission and copyright form etc. would be communicated to you separately (After completion of the registration/payment process).

Please refer to the reviewers' comments in your Microsoft CMT account (log in at: <https://cmt3.research.microsoft.com/ICISSC2025/Submission/Index>), which are intended to help you improve your paper for final publication. You are advised to address the reviewers' comments appropriately in your final paper. The manuscript should be prepared in Springer format.

Thanking You,

Warm Regards

Organizing Chair

ICISSC-2025

+91 9491121470, 9032818166, 7737345621

Please do not reply to this email as it was generated from an email account that is not monitored.

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our [Privacy Statement](#).

Microsoft Corporation

One [Microsoft Way](#)

[Redmond, WA 98052](#)